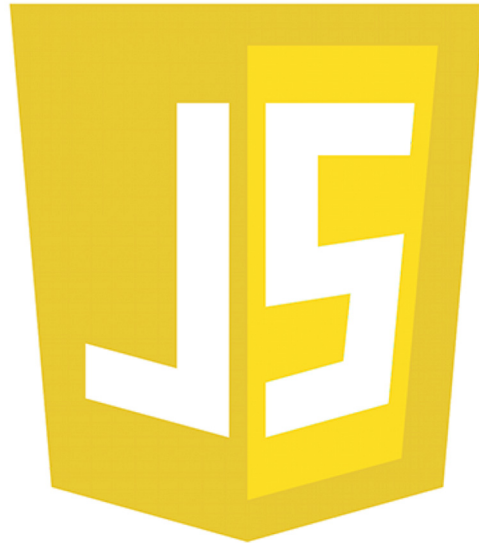


JavaScript

JS



What is *JavaScript*?

JavaScript is an *object-based* (now also *object-oriented*) *scripting language* that integrates with HTML to provide interactive and dynamic functionalities

Created in 1995 by Brendan Eich (subsequently co-founder of the *Mozilla Foundation*), it was born from an alliance between *Sun Microsystems* and *Netscape Communications*; initially, it was called *LiveScript*

A *scripting language* can be seen as a more “versatile” programming language

- Simpler
- More "immediate" (it is interpreted)

Important: JavaScript **is not** Java!

Why to use JavaScript?

JavaScript introduces *interactivity* and “*intelligence*” in web pages.
For example, it is possible to:

- Create and store data on the user's computer
- Process data collected through HTML forms directly on the user's computer
- Make graphics interactive
- Modify pages “on the fly”, according to user input

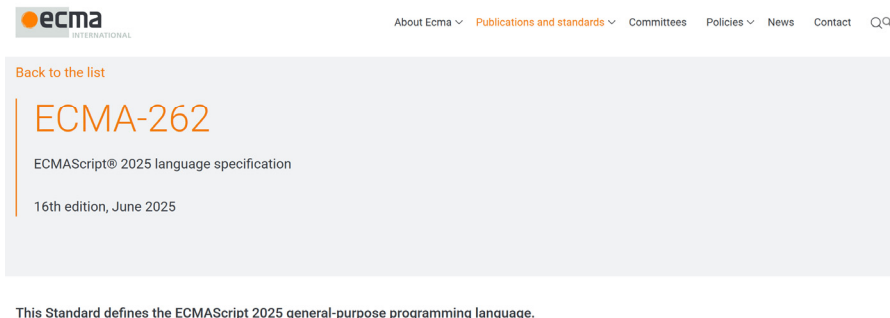
... before JavaScript, gathering and processing user input necessarily required CGI (Common Gateway Interface) scripts, executed on the server (→ communication overhead between client and server)

All web browsers support JavaScript

JavaScript or *ECMAScript*?

The “official” name of JavaScript is *ECMAScript*, developed by *Ecma International*

- Standardized for the first time in 1997 (*ECMAScript 1*), several versions have been progressively created (until *ECMAScript 2025*):
<https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>
- The W3C has developed some *APIs* (Application Programming Interfaces)



Inclusion of JavaScript code in web pages...

```
<!DOCTYPE html>
<html>
  <head>
    .....
    <script>
      <!-- JavaScript code -->
      .....
    </script>
    .....
  </head>
  <body>
    .....
    <script>
      <!-- JavaScript code -->
      .....
    </script>
    .....
  </body>
</html>
```

- Direct inclusion
- The `type="text/javascript"` attribute of the `<script>` tag is mandatory only in HTML 4.01 and XHTML (not in HTML5)

... inclusion of JavaScript code in web pages

```
<!DOCTYPE html>
<html>
  <head>
    .....
    <script src="mycode.js"></script>
    .....
  </head>
  <body>
    .....
    <script src="myothercode.js"></script>
    .....
  </body>
</html>
```

Code defined in
external files

An example

Simple example: form validation



Order Form

Quantity

Total price

Province

Phone number

Let's consider a form that allows us to specify:

- Number of items to buy
- Province
- Phone number

We want to:

- Check the correctness of the data entered (and verify whether they are present)
- In the *Total price* field, display how much the user has to pay, including shipping charges (which depend on the province)

An example

1 – Form definition

```
<form name="orderForm">
  <label for="numItems">Quantity</label>
  <input type="text" name="numItems" id="numItems" size="10">
  <br><br>
  <label for="totalPrice">Total price</label>
  <input type="text" name="totalPrice" id="totalPrice" size="10" readonly>
  <br><br>
  <label for="province">Province</label>
  <input type="text" name="province" id="province" size="2">
  <br><br>
  <label for="phoneNum">Phone number</label>
  <input type="text" name="phoneNum" id="phoneNum" size="15">
  <br><br>
  <input type="button" value="Send">
</form>
```

Since JavaScript is an object-based language, every element of an HTML form is treated as a separate *object*

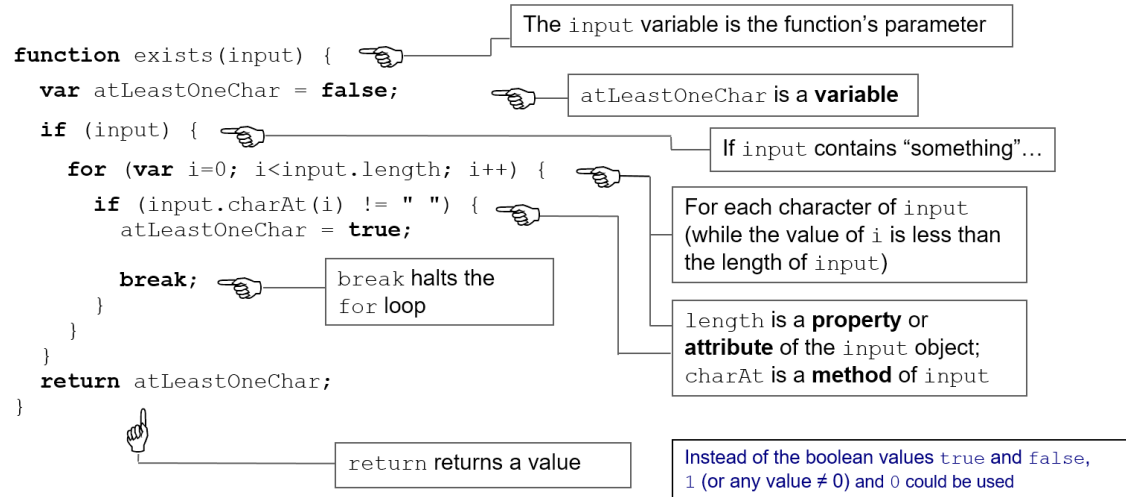
each object has its own data...

An example

2 – Has the user entered the required data?

We want to check whether the user has entered the data for a certain control of the form

We define a JavaScript function that can understand whether a generic string contains at least one character different from a space

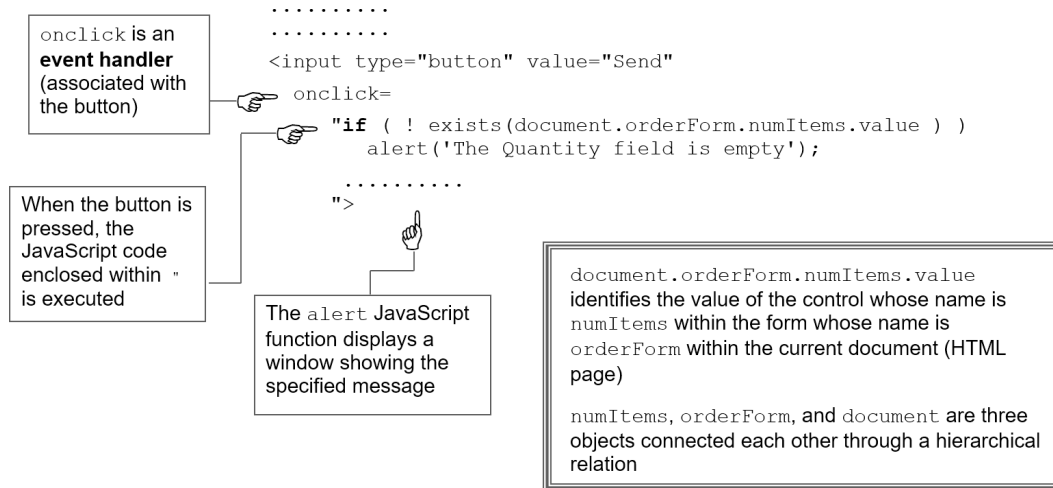


An example

How to use the exists function?

Functions can be invoked by *event handlers* associated with HTML elements

We want to be sure that, before sending the form data to the server, the user has entered a value in the *Quantity* field



An example

3 – Has the user entered an integer number?

We want to check whether a particular data item is an integer number

We define a JavaScript function that is able to recognize integer numbers

```
function integerNumber(input) {  
    var answer = true;  
    if ( ! parseInt(input) ) {  
        answer = false;  
    } else {  
        for (var i=0; i<input.length; i++) {  
            if ((input.charAt(i) != "0")  
                && ( ! parseInt(input.charAt(i))) ) {  
                answer = false;  
                break;  
            }  
        }  
    }  
    return answer;  
}
```

The JavaScript function `parseInt` converts a string into an integer number. If the first character is not a digit, it returns `NaN`, a special string that is treated as the `false` Boolean value; otherwise, it returns the number formed of the digits till the first character that is not a digit (excluded) (e.g., `parseInt("241xy8")` returns 241, while `parseInt("x639")` returns `NaN`).

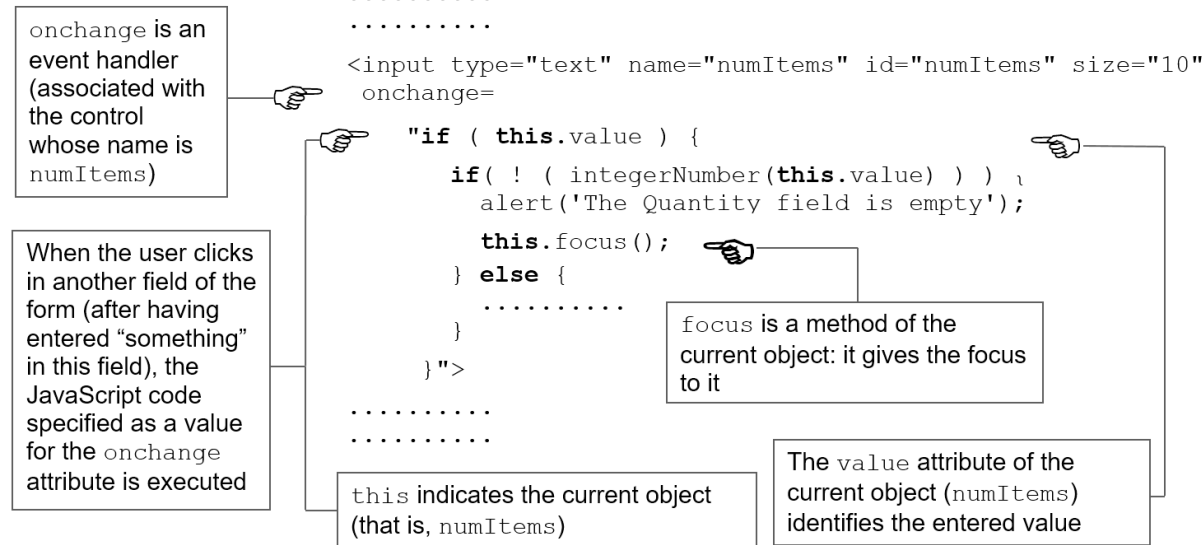
! and && are the NOT and AND logical operators

`(input.charAt(i) != "0")` is necessary to prevent character '0' from being erroneously interpreted as `false` in the second part of the condition (`parseInt(input.charAt(i))` returns the digit in the *i*-th position, `NaN` if the character is not a digit; '0', however, would be treated as `false`...)

An example

How to use the `integerNumber` function?

Example: we want to be sure that the value entered in the *Quantity* field is an integer number



An example

4 – Is the phone number correct?

We want to check whether the entered phone number is an integer number starting with '0' (landline phone numbers in Italy)

We define a JavaScript function able to recognize telephone numbers starting with '0'

```
function phoneNumber(num) {  
    var answer = false;  
    if (num) {  
        if ( (num.charAt(0) == 0) &&  
            integerNumber(num.substring(1,num.length))) )  
            answer = true;  
    }  
    return answer;  
}
```



The `substring(i, j)` method returns a substring of its object (formed of characters from the *i*-th to the *j*-th - 1)

An example

How to use the `phoneNumber` function?

Example: we want to be sure that, before sending the form data to the server, a correct phone number has been entered

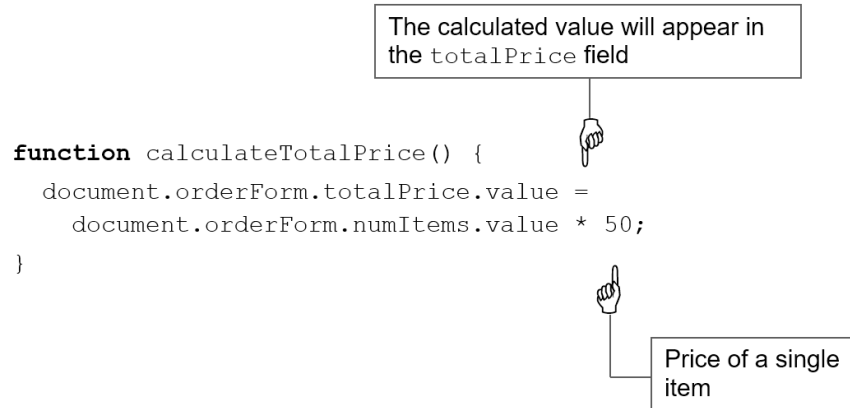
```
.....  
.....  
<input type="button" value="Send"  
  onclick=  
    " .....  
      if (!(phoneNumber(document.orderForm.phoneNum.value)))  
        alert('Incorrect phone number');  
      else if (...)  
        .....  
    ">  
.....  
.....
```

An example

5 – Total price calculation

Once the number of products has been specified, we want the total price to appear in the *Total price* field (for now, without considering shipping charges)

We define a JavaScript function able to calculate the total amount to pay



An example

How to use calculateTotalPrice?

Example:

```
.....  
.....  
<input type="text" name="numItems" id="numItems" size="10"  
  onchange=  
    "if ( this.value ) {  
      if ( ! ( integerNumber(this.value) ) ) {  
        alert('The Quantity field is empty');  
        this.focus();  
      } else {  
        calculateTotalPrice();  
      }  
    } ">  
.....  
.....
```

If the
entered
value is an
integer
number ...



An example

6 – Shipping charges...

When calculating the total price, we want to consider shipping charges as well, which depend on the province

Step 1: Let's define an **object** in which we store shipping charges for each Italian province

```
function shippingCharges() {  
  this.MI = 1;  
  this.PV = 2;  
  this.CO = 2;  
  this.LC = 3;  
  this.VA = 3;  
  this.SO = 4;  
  this.BG = 4;  
  this.LO = 3;  
  .....  
  .....  
}
```



MI, PV, etc. are **attributes** of the shippingCharges object

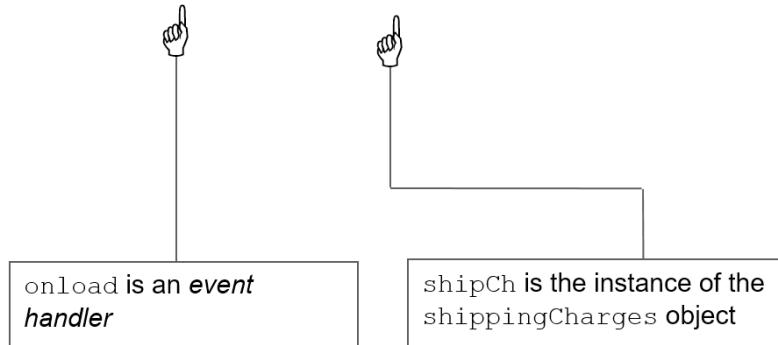
this indicates the current object (i.e., the one being defined)

An example

6 – ... shipping charges

Step2: We want that, when the page is loaded, a new **instance** of the `shippingCharges` object is created

```
<body onload="shipCh = new shippingCharges()">
```



An example

7 – Total price (with shipping charges)

We define a new version of the `calculateTotalPrice` function so that shipping charges are taken into account

```
function calculateTotalPrice(province) {  
    if ( integerNumber(document.orderForm.numItems.value) ) {  
        var temp = 0;  
        if ( exists(document.orderForm.province.value) ) {  
            temp = shipCh[province.toUpperCase()];  
            if ( isNaN(temp) )  
                temp = 0;  
            document.orderForm.totalPrice.value =  
                document.orderForm.numItems.value * 50 + temp;  
        } else {  
            alert('The Quantity field is empty');  
            document.orderForm.numItems.focus();  
        }  
    }  
}
```

If the value of `temp` is NaN (i.e., province does not contain a correct province)

`isNaN` is a predefined function

The method `toUpperCase` converts the characters of its object to uppercase

`shipCh[province.toUpperCase()]` identifies the attribute whose name is given by `province.toUpperCase()` in the `shipCh` instance of the `shippingCharges` object

An example

How to use calculateTotalPrice?

Example:

```
<label for="numItems">Quantity</label>
<input type="text" name="numItems" id="numItems" size="10"
      onchange="calculateTotalPrice (province.value) ">
<br><br>
<label for="province">Province</label>
<input type="text" name="province" id="province" size="2"
      onchange="calculateTotalPrice (this.value) ">
```

An example

8 – Final check of the form

We want to check the form before sending its data to the server

```
function checkForm() {  
    var message = "";  
    if ( ! integerNumber(document.orderForm.numItems.value) )  
        message += "The Quantity field is empty\n";  
  
    if ( ! exists(document.orderForm.province.value) )  
        message += "The Province field is empty\n";  
    else if ( isNaN(shipCh[document.orderForm.province.value.toUpperCase()]) )  
        message += "Wrong province\n";  
  
    if ( ! phoneNumber(document.orderForm.phoneNum.value) )  
        message += "The phone number must be composed of digits only (and start with 0)";  
  
    if ( message != "" )  
        alert(message);  
    else  
        document.orderForm.submit(); // data are sent to the server  
}
```

An example

How to use the checkForm function?

```
<input type="button" value="Send" onclick="checkForm()" ">
```

Example:

The image shows two components: a web form and a validation error dialog. The web form, titled "Order Form", has a light green background and contains four input fields: "Quantity" with the value "10", "Total price" with the value "500", "Province" which is empty, and "Phone number" with the value "5382123456". A "Send" button is at the bottom. To the right, a white dialog box with a grey border displays two error messages: "The Province field is empty" and "The phone number must be composed of digits only (and start with 0)". An "OK" button is at the bottom right of the dialog.

Order Form

Quantity

Total price

Province

Phone number

The Province field is empty
The phone number must be composed of digits only (and start with 0)

JavaScript *objects* and the *DOM*

An object is composed of *properties (attributes)* and *methods*

JavaScript programmers can define their own objects

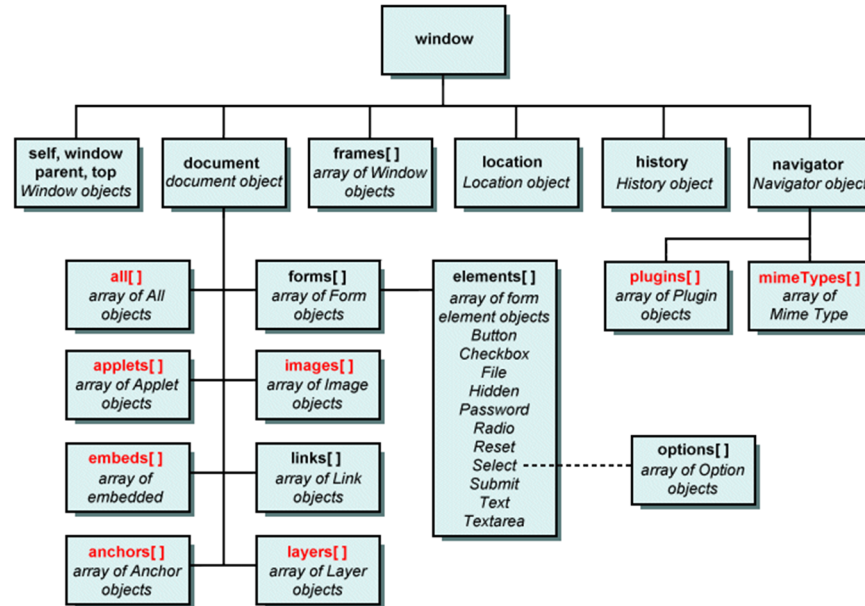
Often, however, they will use “built-in” objects...

The set of predefined objects, hierarchically structured, is called the *Document Object Model (DOM)*

The DOM is technically **separated from JavaScript** (it does not pertain to the specification of the language)

- It is associated with the browser
- Netscape and Microsoft developed different DOMs

Basic DOM



Event handlers...

JavaScript programs are typically *event-driven*

I.e., they are controlled by “events”

An *event* occurs in a web page, often as a result of a user action

Examples of events: press of a button, resize of the browser window, rollover, ...

An *event handler* is always associated with an event

- It defines the JavaScript code that will be executed when the event occurs (examples: *onclick*, *onchange*, *onload*, ...)
- The possible events that can occur in a web page **are part of the DOM**

... event handlers

Event	Event handler	Description (the event occurs when...)
click	onclick	The user presses the left mouse button
change	onchange	The user changes the content of a form field
focus	onfocus	The user gives the focus to a field
blur	onblur	The user removes the focus from a field
mouseover	onmouseover	The user moves the mouse cursor over a certain element
mouseout	onmouseout	The user takes the mouse cursor away from a certain element
select	onselect	The user selects an input field in a form
submit	onsubmit	The user sends form data to the server
resize	onresize	The user changes the size of the browser window
load	onload	The user loads a new page
unload	onunload	The user loads another page
reset	onreset	The user resets all the fields of a form

Creating objects...

```
<!DOCTYPE html>
<html>
  <head>
    <title>Creating objects (1)</title>
    <script>
      function car(man, mod, licpl) {
        this.manufacturer = man;
        this.model = mod;
        this.licenseplate = licpl;
        this.displayCar = displayCar;
      }
      function displayCar() {
        document.write("The manufacturer is: " +
          this.manufacturer + "<br>");
        document.write("The model is: " +
          this.model + "<br>");
        document.write("The licence plate is: " +
          this.licenseplate);
        document.write("<br><br>");
      }
    </script>
  </head>
  <body>
    <script>
      myCar = new car("Fiat", "Panda", "AC123QZ");
      myOtherCar = new car("Ford", "Fiesta", "BR123AM");
      myCar.displayCar();
      myOtherCar.displayCar();
    </script>
  </body>
</html>
```

Alternative solution

```
function car(man, mod, licpl) {
  this.manufacturer = man;
  this.model = mod;
  this.licenseplate = licpl;
  this.displayCar = function () {
    document.write("The manufacturer is: " +
      this.manufacturer + "<br>");
    document.write("The model is: " +
      this.model + "<br>");
    document.write("The licence plate is: " +
      this.licenseplate);
    document.write("<br><br>");
  }
}
```

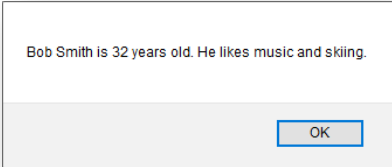
The manufacturer is: Fiat
The model is: Panda
The licence plate is: AC123QZ

The manufacturer is: Ford
The model is: Fiesta
The licence plate is: BR123AM

Objects

... creating objects...

```
<!DOCTYPE html>
<html>
  <head>
    <title>Creating objects (2)</title>
    <script>
      function Person(first, last, age, gender, interests) {
        this.name = {
          first: first,
          last: last
        };
        this.age = age;
        this.gender = gender;
        this.interests = interests;
        this.bio = function() {
          var pronoun;
          if (gender == 'male') pronoun = "He"; else pronoun = "She";
          alert(this.name.first + ' ' + this.name.last + ' is ' + this.age + ' years old. ' +
            pronoun + ' likes ' + this.interests[0] + ' and ' + this.interests[1] + '. ');
        };
      }
    </script>
  </head>
  <body>
    <script>
      var person1 = new Person('Bob', 'Smith', 32, 'male', ['music', 'skiing']);
      person1.bio();
    </script>
  </body>
</html>
```



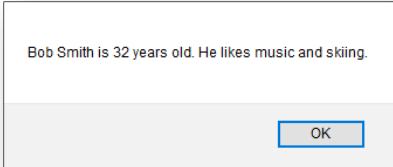
Bob Smith is 32 years old. He likes music and skiing.

OK

Objects

... creating objects

```
<!DOCTYPE html>
<html>
  <head>
    <title>Creating objects (3)</title>
    <script>
      var Person = {
        name: {
          first: 'Bob',
          last: 'Smith'
        },
        age: 32,
        gender: 'male',
        interests: ['music', 'skiing'],
        bio: function() {
          var pronoun;
          if (this.gender == 'male') pronoun = "He"; else pronoun = "She";
          return this.name.first + ' ' + this.name.last + ' is ' + this.age + ' years old. ' +
            pronoun + ' likes ' + this.interests[0] + ' and ' + this.interests[1] + '. ';
        }
      }
    </script>
  </head>
  <body>
    <script>
      alert(Person.bio());
    </script>
  </body>
</html>
```



Bob Smith is 32 years old. He likes music and skiing.

OK

“Recent” JavaScript

Classes, *let*, *const*

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    document.write(this.name + ' makes a noise.');
```

```
class Dog extends Animal {
  constructor(name) {
    super(name);
  }

  speak() {
    document.write(this.name + ' barks.');
```

```
let d = new Dog('Mitzie');
d.speak(); // Mitzie barks
```

- `myvariable = 'Hello'`
Scope: global
- `var myvariable = 'Hello'`
Scope: enclosing function
- `let myvariable = 'Hello'`
Scope: enclosing block
- `const myconst = 'Hello'`
Scope: enclosing block (unmodifiable value)